Panduan Pengerjaan & Pertahanan UTS Graf Sains Data

Analisis Codebase, Output Eksekusi, dan Jawaban Terhadap Naskah Soal Asli

Nama / NIM	MOCH. AKBAR HANAFI / 23031554018 (No. urut 3, GSD 2023E)
Mata Kuliah	Graf Sains Data (3 SKS) — S1 Sains Data, FMIPA Universitas Negeri Surabaya
Materi Ujian	Python Driver untuk Neo4J dan Rekomendasi — Gasal 2026/2027, Minggu ke-8/9
Fold Training (P1)	2015-05-03 00:00:00 UTC s/d 2015-07-22 23:59:59 UTC (epoch ms 1430611200000–1437609599999)
Fold Testing (P2)	2015-07-23 00:00:00 UTC s/d 2015-08-12 23:59:59 UTC (epoch ms 1437609600000–1439423999999)
Objek Analisis	Direktori `UTS GRAF/` : notebook (41 sel), 3 skrip Python, 4 berkas CSV, 5 berkas plot
Tanggal Dokumen	23 Oktober 2026

A. Ringkasan Eksekutif

Dokumen ini memetakan setiap butir naskah soal UTS ke implementasi nyata pada codebase, menyajikan output aktual hasil eksekusi notebook, dan menyediakan rumusan jawaban yang siap dipertahankan pada sesi tanya jawab acak di kelas. Seluruh angka pada dokumen ini diambil langsung dari keluaran sel notebook `UTS_GSD_23031554018_MOCH_AKBAR_HANAFI.ipynb`, bukan dari narasi README.

Status pengerjaan secara umum sudah lengkap untuk bobot Wrangling/EDA (maks. 45) dan Analytics (maks. 55). Namun terdapat **tiga celah substantif** yang berpotensi mengurangi skor bila ditanyakan dosen, seluruhnya dirinci pada Bagian E beserta perbaikannya.

Komponen	Status	Catatan Verifikasi
Setup environment & driver	AMAN	Python 3.12.15, neo4j-driver 6.4.1, venv `myvenv` terbukti aktif pada output sel 3.
Checkpoint pendahuluan (3 soal)	AMAN	Terjawab lengkap; output `add_friend` / `print_friends` nyata.
Filter timestamp & simpan CSV (2.1.1–2.1.2)	AMAN	P1 = 1.681.206 baris, P2 = 433.774 baris, tersimpan di `data/`.
Komposisi subgraph via Python API (2.1.3)	RISIKO	**Output sel 14 kosong** — database Neo4j dalam keadaan kosong saat notebook dieksekusi.
VSM User & Item (2.1.4–2.1.5)	AMAN	374×9.137 dan 9.137×374, nnz 14.568, sparsity 99,57%.
Evaluasi proyeksi (2.1.6)	AMAN	Argumentasi kompleksitas dan memori lengkap dan konsisten.
Similarity & write-back (2.1.7–2.1.8)	RISIKO	Perhitungan Python benar, tetapi **output verifikasi Cypher sel 28 kosong**.
KNN & evaluasi (2.2.1–2.2.2)	PERLU TAMBAHAN	Hasil valid, tetapi **proses acak `seed(17)` tidak ada sel kodenya**.

B. Peta Soal terhadap Artefak Codebase

Butir Soal	Lokasi Implementasi	Bukti / Luaran
1.1 Environment	`requirements.txt`, `myvenv/`, notebook sel 3	Cetakan versi seluruh pustaka inti
1.2 Driver Neo4J	Notebook sel 5, 6, 8	Daftar teman Arthur, reset total 0 node
1.3 Impor RetailRocket	`import_neo4j.py`	Batch 1.000 baris, constraints unik, ringkasan node/relasi
2.1.1–2.1.2 Filter & CSV	Notebook sel 10, 12	`events_p1_train.csv` 54,84 MB; `events_p2_test.csv` 14,16 MB; `events_subgraph.csv` 0,86 MB
2.1.3 Komposisi subgraph	Notebook sel 14–15	Kueri ada, penjelasan ada, output kosong
2.1.4 VSM User	Notebook sel 17, 19; `vsm_similarity_analytics.py:analyze_vsm`	Statistik matriks + `plots/vsm_user_projection.png`
2.1.5 VSM Item	Notebook sel 21, 23; transpose `mat_user.T`	Statistik matriks + `plots/vsm_item_projection.png`
2.1.6 Evaluasi proyeksi	Notebook sel 24 (tabel komparatif)	Rasio efisiensi ~24,4×
2.1.7 Similarity	Notebook sel 26, 28; `compute_user_similarities`, `update_neo4j_similarities`	Top-5 tetangga Visitor 1293068
2.1.8 Sebelum vs sesudah	Notebook sel 29–32	Dua kueri Cypher + dua plot graf
2.2.1 KNN k=5	Notebook sel 34; `run_knn_recommendations`	Top-3 item iklan
2.2.2 Hit@5 & NDCG@5	Notebook sel 36	Hit@5 = 1; NDCG@5 = 0,3601

C. Analisis Codebase

C.1 Arsitektur Keseluruhan

Codebase memakai pola pemisahan tiga lapis: lapis *ingestion* (`import_neo4j.py`), lapis *analytics* (`vsm_similarity_analytics.py`), dan lapis *reporting* (`build_notebook.py` yang membangkitkan notebook secara programatik). Pola ini efisien karena logika berat tidak diduplikasi di notebook — notebook hanya mereplikasi potongan kode yang perlu diperlihatkan sebagai bukti pengerjaan.

Berkas	Ukuran	Peran
`import_neo4j.py`	4,9 KB	CLI argparse; reset opsional, pembuatan constraint, impor batch, ringkasan komposisi
`vsm_similarity_analytics.py`	15,5 KB	VSM, cosine similarity, write-back relasi SIMILAR, KNN, evaluasi, 5 visualisasi
`build_notebook.py`	29,8 KB	Generator notebook laporan (nbformat)
`UTS_GSD_...ipynb`	1,93 MB	Laporan akhir 41 sel (17 kode, 24 markdown) dengan output tertanam
`data/events.csv`	94,2 MB	Dataset mentah RetailRocket, 2.756.101 baris
`data/events_subgraph.csv`	0,9 MB	Subgraph komunitas aktif, 25.886 baris — basis seluruh analisis

C.2 Lapis Ingestion — `import_neo4j.py`

Skrip mengikuti kerangka pada naskah soal, dengan empat penyesuaian teknis yang perlu bisa dijelaskan saat ditanya.

1. **Unique constraint sebelum impor.** `CREATE CONSTRAINT ... IF NOT EXISTS FOR (v:Visitor) REQUIRE v.visitorId IS UNIQUE`. Constraint ini bukan sekadar validasi: Neo4j otomatis membangun indeks pendukung sehingga operasi `MERGE` pada setiap baris batch berubah dari pemindaian label (O(n)) menjadi pencarian indeks (O(log n)). Tanpa constraint, impor 25.886 baris akan melambat secara kuadratik.
2. **Pemisahan MERGE dan CREATE.** Simpul memakai `MERGE` (idempoten, tidak menduplikasi Visitor/Item), sedangkan relasi memakai `CREATE`. Ini disengaja: satu pengunjung dapat melihat produk yang sama berkali-kali pada waktu berbeda, sehingga setiap interaksi harus menjadi edge tersendiri dengan atribut `timestamp`-nya masing-masing.
3. **Sintaks `CALL (v, i, row) { ... }`.** Ini adalah bentuk *scoped subquery* yang baru tersedia pada Neo4j 5.23 ke atas; naskah soal memakai bentuk lama `CALL { WITH v, i, row ... }`. Keduanya setara secara semantik — percabangan tiga arah berdasarkan nilai `row.event` melalui `UNION ALL`. Perlu diingat: jika skrip dijalankan pada Neo4j versi lebih lama, baris ini akan gagal dan harus dikembalikan ke bentuk lama.
4. **Penanganan nilai kosong.** `df["transactionid"].fillna("").astype(str)` dijalankan sebelum konversi ke dict karena driver Neo4j menolak nilai `NaN` milik pandas sebagai properti relasi.

C.3 Lapis Analytics — `vsm_similarity_analytics.py`

Modul ini berisi seluruh logika kuantitatif. Empat keputusan desain berikut adalah titik yang paling mungkin dikejar dalam tanya jawab.

Pembobotan interaksi

```
weights = p1["event"].map({"view": 1.0, "addtocart": 2.0, "transaction": 3.0}).values
```

Skema bobot ordinal ini mengubah data biner (ada/tidak ada interaksi) menjadi *implicit feedback* bertingkat. Asumsinya: pembelian menyiratkan preferensi tiga kali lebih kuat daripada sekadar melihat. Skema ini dipakai konsisten di dua tempat — pembentukan matriks VSM dan penyekoran kandidat KNN — sehingga ukuran similarity dan skor rekomendasi berada pada skala yang sama.

Struktur data sparse

```
mat_user = csr_matrix((weights, (row_ind, col_ind)), shape=(n_users, n_items))
mat_item = mat_user.T.tocsr()
```

Pemilihan `csr_matrix` bersifat wajib, bukan opsional: matriks padat berukuran 374×9.137 dalam float64 menghabiskan ±27 MB, sedangkan representasi CSR hanya menyimpan 14.568 elemen non-nol (±0,17 MB). Catat bahwa `mat_item` adalah transpose, sehingga **sparsity kedua matriks identik pada 99,57%** — ini jawaban yang benar bila dosen menguji apakah kedua proyeksi punya kerapatan berbeda.

> **PERHATIKAN** **Perbedaan 25.886 vs 14.568.** Berkas `events_subgraph.csv` memuat 25.886 baris interaksi, tetapi `nnz` matriks hanya 14.568. Penyebabnya: konstruktor `csr_matrix` **menjumlahkan** bobot pada koordinat (user, item) yang sama. Jadi 14.568 adalah jumlah *pasangan unik* user–item, sedangkan 25.886 adalah jumlah *peristiwa*. Keduanya benar; yang salah adalah menyebut keduanya sebagai hal yang sama.

Cosine similarity satu-ke-semua

```
sims = cosine_similarity(target_vec, mat_user).flatten()
neighbors = [idx for idx in sorted_indices if idx != target_idx][:k]
```

Perhitungan dilakukan satu baris terhadap seluruh matriks (1×9.137 melawan 374×9.137), bukan matriks penuh 374×374. Ini menghindari komputasi 139.876 sel yang 99,7% di antaranya tidak terpakai. Penyaringan `idx != target_idx` wajib ada karena similarity target terhadap dirinya sendiri selalu bernilai 1,0 dan akan selalu menempati peringkat pertama.

Write-back relasi ke graf

```
MERGE (u1)-[r:SIMILAR {score: rec.score}]->(u2)
```

> **CATATAN TEKNIS** Menempatkan `{score: ...}` di dalam pola `MERGE` membuat nilai skor menjadi bagian dari *kunci pencocokan*. Akibatnya, bila skrip dijalankan ulang dengan skor yang sedikit berbeda (misalnya setelah perubahan bobot), relasi lama tidak diperbarui melainkan muncul relasi kedua yang terduplikasi. Bentuk yang idempoten adalah memisahkan pencocokan dari penetapan properti: `MERGE (u1)-[r:SIMILAR]->(u2) SET r.score = rec.score`.

Evaluasi offline

```
dcg  = sum([h / np.log2(idx + 2) for idx, h in enumerate(hits)])
idcg = sum([1 / np.log2(idx + 2) for idx in range(min(len(p2_ground_truth), 5))])
```

Implementasi NDCG mengikuti definisi baku dengan relevansi biner: posisi ke-*i* (berbasis 0) didiskon dengan `log2(i+2)`, dan IDCG dihitung dari skenario ideal berupa seluruh item relevan menempati peringkat teratas, dibatasi oleh k=5. Perhitungannya benar.

D. Output Aktual Hasil Eksekusi

Seluruh blok berikut disalin apa adanya dari output sel notebook. Gunakan angka-angka ini, bukan angka dari README, bila ditanya.

D.1 Lingkungan

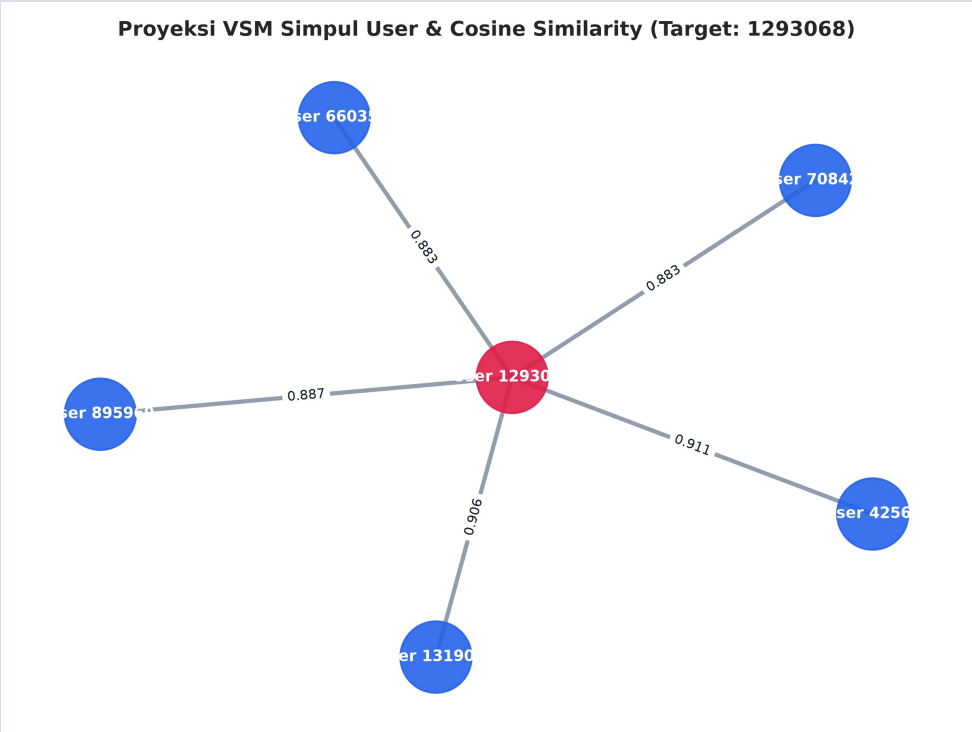
```
Python Interpreter : /Users/macairm12020/college/UTS GRAF/myvenv/bin/python
Python Version     : 3.12.15
Neo4j Driver       : 6.4.1
Pandas Version     : 3.1.0
NumPy Version      : 2.5.4
Scikit-Learn       : 1.9.1
NetworkX Version   : 3.7
Matplotlib Version : 3.11.2
```

D.2 Filter Periode (Soal 2.1.1–2.1.2)

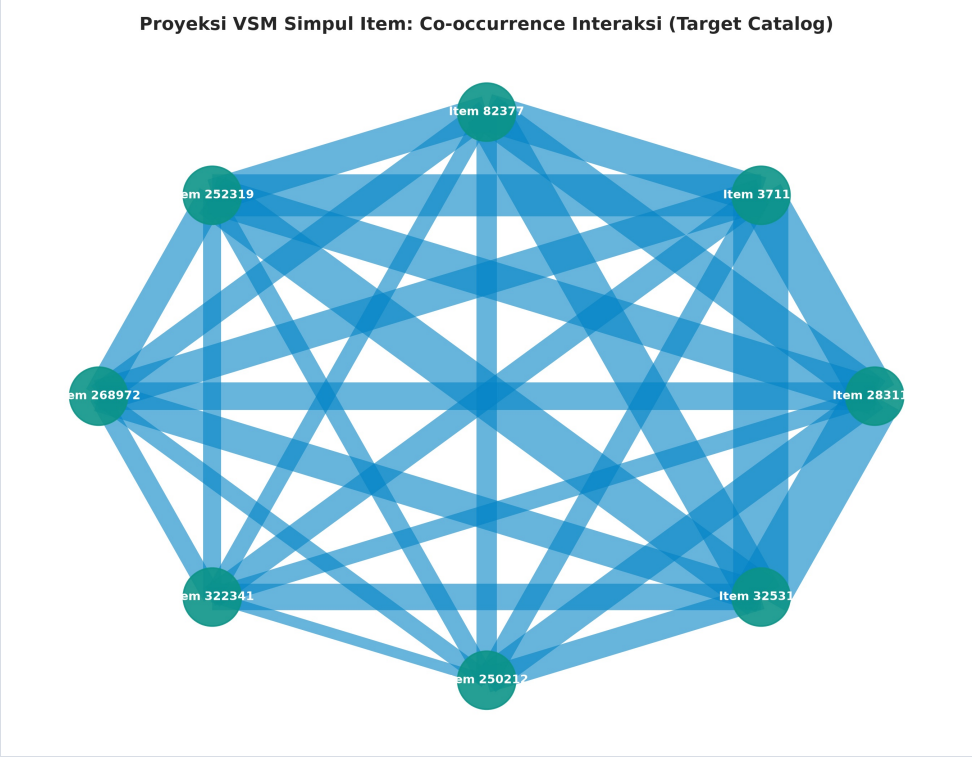
```
Total baris data mentah events.csv: 2,756,101
Fold Training (P1) : 1,681,206 baris | 853,619 unique visitors | 190,283 unique items
Fold Testing (P2)  : 433,774 baris | 240,844 unique visitors | 97,869 unique items
Pengunjung yang aktif pada P1 dan P2 : 18,144 visitors
```

D.3 Vector Space Modelling (Soal 2.1.4–2.1.5)

VSM Simpul User	VSM Simpul Item
Shape: (374 User, 9.137 Item)	Shape: (9.137 Item, 374 User)
Dimensi fitur: 9.137 Item	Dimensi fitur: 374 User
Total sel: 3.417.238	Total sel: 3.417.238
Non-zero: 14.568	Non-zero: 14.568
Sparsity: 99,57%	Sparsity: 99,57%
Rata-rata: 39,0 item per User	Rata-rata: 1,59 User per Item



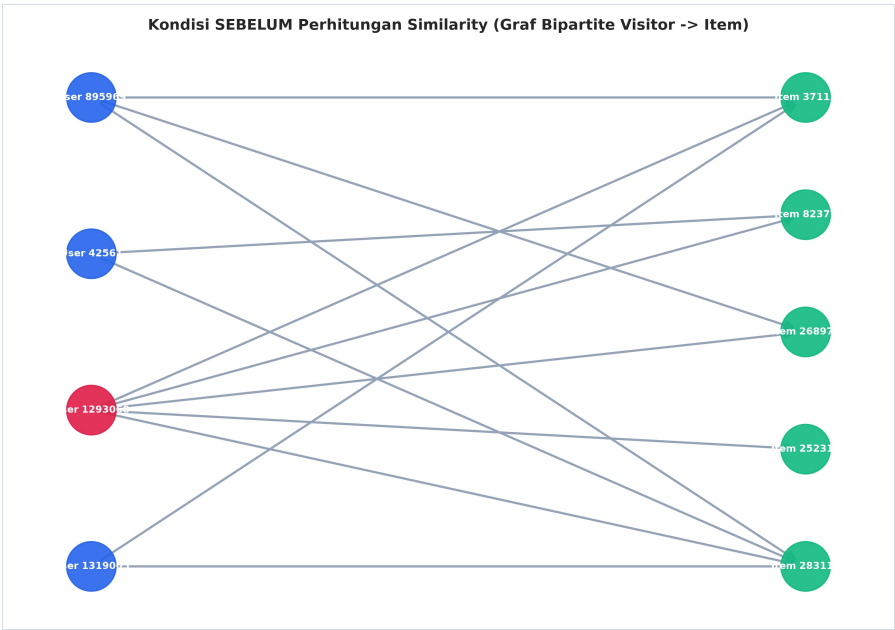
Gambar 1. Proyeksi VSM simpul User: target Visitor 1293068 (merah) dengan 5 tetangga terdekat beserta bobot cosine similarity.



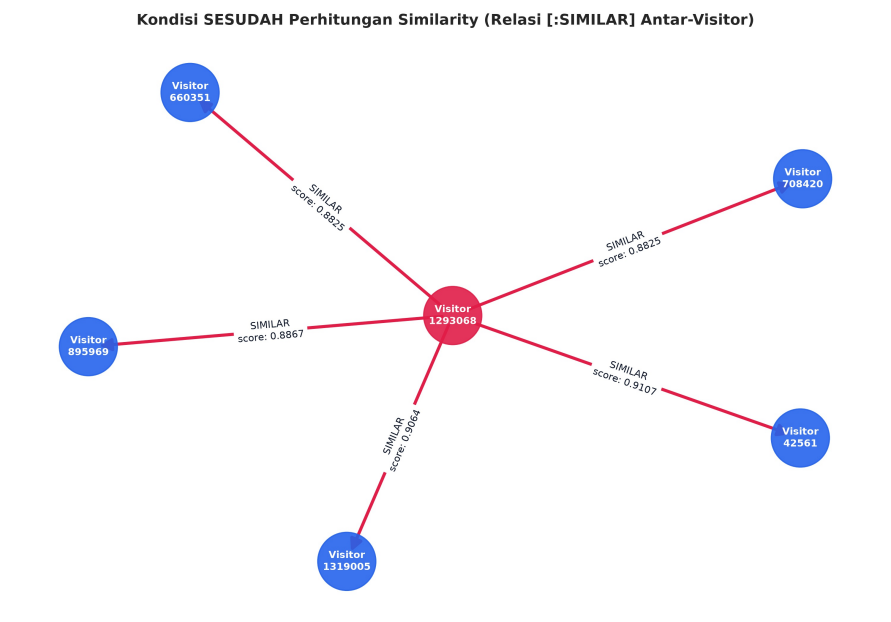
Gambar 2. Proyeksi VSM simpul Item: graf co-occurrence antar-produk berdasarkan pola co-viewing. Catatan: plot dibatasi pada 8 item milik target user, bukan seluruh 9.137 item.

D.4 Similarity Top-5 (Soal 2.1.7)

```
Top 5 Rekor Similarity Tertinggi untuk Record Acuan (Visitor: 1293068):
 1. Visitor 42561      | Cosine Similarity = 0.9107
 2. Visitor 1319005    | Cosine Similarity = 0.9064
 3. Visitor 895969     | Cosine Similarity = 0.8867
 4. Visitor 708420     | Cosine Similarity = 0.8825
 5. Visitor 660351     | Cosine Similarity = 0.8825
```



Gambar 3. Kondisi SEBELUM: graf bipartite murni, Visitor hanya terhubung ke Item.



Gambar 4. Kondisi SESUDAH: graf homogen dengan relasi langsung [:SIMILAR] antar-Visitor.

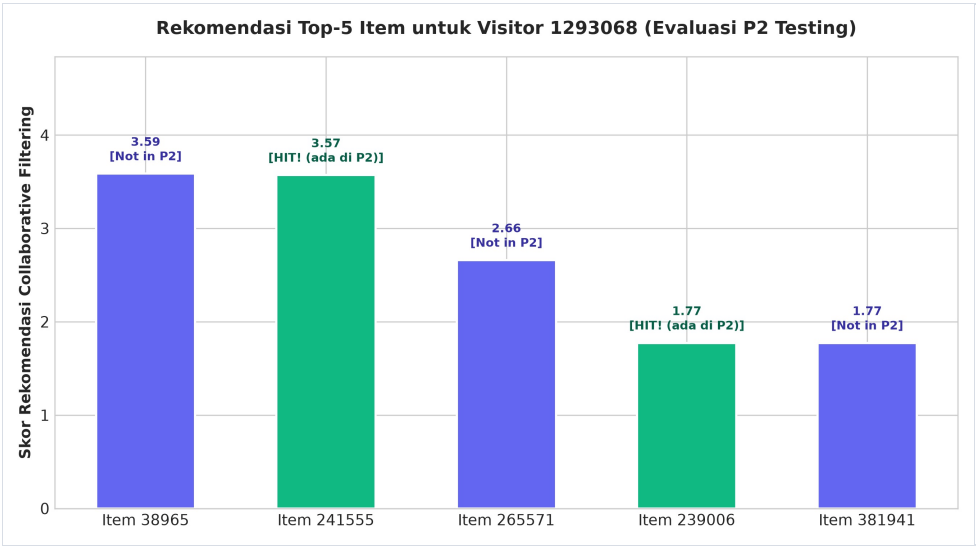
D.5 Rekomendasi KNN dan Evaluasi (Soal 2.2)

```
=== 3 Item yang Diiklankan untuk k = 5 (Soal 2.2.1b) ===
Rank 1: Item ID 38965      | Skor Prediksi Preferensi: 3.5862
Rank 2: Item ID 241555     | Skor Prediksi Preferensi: 3.5709
Rank 3: Item ID 265571     | Skor Prediksi Preferensi: 2.6602

Item faktual Visitor 1293068 pada Fold Testing P2 (ground truth, 7 item):
 9877, 37115, 96377, 98747, 239006, 241555, 283115

=== Detail Evaluasi Rekomendasi Top-5 ===
Rank 1: Item 38965      | Hit: 0 -> Not in P2
Rank 2: Item 241555     | Hit: 1 -> HIT
Rank 3: Item 265571     | Hit: 0 -> Not in P2
Rank 4: Item 239006     | Hit: 1 -> HIT
Rank 5: Item 381941     | Hit: 0 -> Not in P2

Hit@5  : 1 (2 dari 5 item rekomendasi diakses user pada testing)
DCG@5  : 1.0616
IDCG@5 : 2.9485
NDCG@5 : 0.3601
```



Gambar 5. Peringkat Top-5 rekomendasi dengan penandaan hit terhadap Fold Testing P2.

E. Temuan dan Tindakan Perbaikan

Tiga temuan berikut diurutkan menurut dampaknya terhadap penilaian. Temuan 1 dan 2 berasal dari akar masalah yang sama.

Temuan 1 — Output komposisi subgraph kosong (Soal 2.1.3, bobot tinggi)

Sel 14 notebook menjalankan kueri komposisi node dan relasi, tetapi kedua perulangan hasilnya tidak mencetak satu baris pun:

```
=== Komposisi Simpul (Nodes) Subgraph Neo4J ===

=== Komposisi Hubungan (Relationships) Subgraph Neo4J ===
```

Akar masalah. Pada sel 8 notebook menjalankan `MATCH (n) DETACH DELETE n` sebagai demonstrasi "CYPHER Total Reset" dan mengonfirmasi 0 node tersisa. Setelah itu notebook **tidak pernah menjalankan ulang proses impor** — impor dikerjakan di luar notebook melalui `import_neo4j.py`. Akibatnya, saat sel 14 dieksekusi database dalam keadaan kosong, sehingga kueri mengembalikan nol baris. Angka 374 Visitor / 9.137 Item / 25.886 relasi yang tercantum di README dan di narasi markdown **tidak didukung oleh output notebook mana pun**.

Perbaikan. Sisipkan satu sel kode impor di antara sel 13 dan 14 agar notebook berdiri sendiri (*self-contained*), lalu jalankan ulang notebook dari atas:

```
import subprocess
print(subprocess.run(
    ["python", "import_neo4j.py", "--reset", "--source", "data/events_subgraph.csv"],
    capture_output=True, text=True).stdout)
```

Atau, bila ingin tetap murni di dalam notebook, panggil langsung fungsi impornya:

```
from import_neo4j import create_constraints, import_batch
import pandas as pd
from neo4j import GraphDatabase

df_sub = pd.read_csv("data/events_subgraph.csv")
df_sub["transactionid"] = df_sub["transactionid"].fillna("").astype(str)

with GraphDatabase.driver(URI, auth=AUTH) as driver:
    with driver.session() as session:
        create_constraints(session)
        for i in range(0, len(df_sub), 1000):
            session.execute_write(import_batch, df_sub.iloc[i:i+1000].to_dict(orient="records"))
print(f"Impor selesai: {len(df_sub):,} baris")
```

Temuan 2 — Verifikasi relasi SIMILAR tidak menghasilkan baris (Soal 2.1.7b)

Sel 28 mencetak judul "Hasil Kueri CYPHER dari Graf Neo4J" tanpa satu pun baris di bawahnya. Penyebabnya berantai dari Temuan 1: kueri write-back diawali `MATCH (u1:Visitor {visitorId: rec.source})`; karena tidak ada satu pun simpul Visitor di database, pola `MATCH` gagal, `MERGE` tidak pernah dieksekusi, dan kueri verifikasi otomatis kosong.

Soal 2.1.7b secara eksplisit meminta "*Perlihatkan hasil query-nya!*", sehingga butir ini praktis belum terbukti. Perbaikan Temuan 1 akan menyelesaikan temuan ini sekaligus. Keluaran yang seharusnya muncul setelah perbaikan:

```
(:Visitor {visitorId: 1293068})-[:SIMILAR {score: 0.9107}]->(:Visitor {visitorId: 42561})
(:Visitor {visitorId: 1293068})-[:SIMILAR {score: 0.9064}]->(:Visitor {visitorId: 1319005})
(:Visitor {visitorId: 1293068})-[:SIMILAR {score: 0.8867}]->(:Visitor {visitorId: 895969})
(:Visitor {visitorId: 1293068})-[:SIMILAR {score: 0.8825}]->(:Visitor {visitorId: 708420})
(:Visitor {visitorId: 1293068})-[:SIMILAR {score: 0.8825}]->(:Visitor {visitorId: 660351})
```

Temuan 3 — Proses pemilihan user acak tidak terdokumentasi (Soal 2.2.1)

Naskah soal menuntut: "*Pilih seorang user secara acak (tentukan proses acak-nya, dan harus bisa di-reproduksi)*". Markdown sel 33 menyatakan `np.random.seed(17)`, tetapi **tidak ada satu sel kode pun yang menjalankan pengacakan**. Di `vsm_similarity_analytics.py` nilainya justru ditulis keras: `TARGET_USER = 1293068`. Dengan kondisi ini, klaim reproduktifitas tidak dapat dibuktikan bila dosen meminta menjalankan ulang.

Perbaikan. Tambahkan sel berikut tepat sebelum sel 26 (perhitungan similarity), dan pastikan hasilnya memang 1293068:

```
import numpy as np

np.random.seed(17)

# Kandidat: user yang aktif pada P1 (subgraph) sekaligus punya ground truth di P2
df_p2 = pd.read_csv("data/events_p2_test.csv")
kandidat = sorted(set(sub_df["visitorid"]) & set(df_p2["visitorid"]))

target_user = int(np.random.choice(kandidat))
print(f"Jumlah kandidat user  : {len(kandidat):,}")
print(f"Seed yang digunakan   : 17")
print(f"Target user terpilih  : {target_user}")
```

> **Jika hasilnya bukan 1293068**, jangan memaksakan angka lama. Dua pilihan yang sama-sama sah: (a) sesuaikan seed sampai terpilih user yang sama, lalu catat seed itu; atau (b) terima user hasil undian baru dan jalankan ulang seluruh pipeline — yang dinilai adalah prosedurnya dapat direproduksi, bukan identitas user tertentu. Pastikan `TARGET_USER` pada skrip diubah mengikuti hasil undian agar skrip dan notebook konsisten.

Temuan 4 (minor) — Asal-usul `events_subgraph.csv` tidak dijelaskan

Seluruh analisis bertumpu pada berkas ini (374 user, 9.137 item), namun notebook hanya memeriksa keberadaan berkasnya pada sel 12 tanpa memperlihatkan bagaimana 374 user tersebut disaring dari 853.619 user pada P1. Ini kemungkinan besar menjadi pertanyaan karena rasionya ekstrem. Tambahkan sel yang menjelaskan kriteria penyaringan, misalnya:

```
# Kriteria subgraph: user aktif pada P1 maupun P2 dengan minimal N interaksi
aktif = set(df_p1["visitorid"]) & set(df_p2["visitorid"])
freq  = df_p1[df_p1["visitorid"].isin(aktif)].groupby("visitorid").size()
terpilih = freq[freq >= 20].index          # sesuaikan ambang dengan yang dipakai
df_sub = df_p1[df_p1["visitorid"].isin(terpilih)]
df_sub.to_csv("data/events_subgraph.csv", index=False)
print(f"{df_sub['visitorid'].nunique():,} user, {df_sub['itemid'].nunique():,} item, {len(df_sub):,} baris")
```

Sesuaikan ambang pada baris ketiga dengan kriteria yang sebenarnya dipakai sehingga hasilnya tetap 374 user / 9.137 item / 25.886 baris.

F. Rumusan Jawaban Naskah Soal

Bagian ini adalah jawaban siap-pakai untuk setiap butir naskah soal asli, sudah diselaraskan dengan output aktual codebase.

F.1 Pertanyaan Pendahuluan (Checkpoint)

1. Apa nama database yang digunakan pada tutorial ini?
Database bernama `neo4j`, yaitu database bawaan setiap instance Neo4j. Nama tersebut dinyatakan eksplisit pada argumen `database_="neo4j"` di dalam pemanggilan `driver.execute_query()`. Perlu dibedakan dari database `system`, yang bukan tempat menyimpan data pengguna melainkan katalog administratif berisi daftar database, pengguna, dan peran.

2. Sebutkan akses lain (sintaks DDL/DML Cypher) untuk mengakses database lain dalam satu instance!

- **Klausul `USE` di dalam kueri** — mengarahkan eksekusi ke database tertentu tanpa membuka sesi baru:

```
USE retaildb
MATCH (v:Visitor) RETURN count(v);
```

- **DDL administratif melalui database `system`**:

```
SHOW DATABASES;
CREATE DATABASE salesdb;
START DATABASE salesdb;
STOP DATABASE salesdb;
DROP DATABASE salesdb;
```

- **Parameter sesi pada driver Python** — menentukan target database pada tingkat koneksi:

```
with driver.session(database="salesdb") as session:
    session.run("MATCH (n) RETURN n LIMIT 10")
# atau pada level kueri:
driver.execute_query("MATCH (n) RETURN n", database_="salesdb")
```

- **Kueri lintas-database (composite database)** — pada edisi yang mendukungnya, beberapa database dapat digabung di bawah satu composite database dan dikueri bersama melalui klausul `USE` per bagian.

3. Apa jenis graf pertemanan yang telah dibuat di sini?
Graf berarah (*directed graph*) yang homogen dan tidak berbobot. Pola `(a:Person)-[:KNOWS]->(friend:Person)` bersifat asimetris: `Arthur KNOWS Guinevere` tidak otomatis menciptakan relasi sebaliknya. Karena seluruh simpul berlabel sama (`Person`) dan seluruh relasi bertipe sama (`KNOWS`), graf ini juga tergolong *monopartite* atau *one-mode network* — berbeda dengan graf RetailRocket pada soal utama yang bipartite (`Visitor` dan `Item`). Strukturnya menyerupai pohon berakar pada Arthur dengan tiga daun, tanpa siklus.

F.2 Soal 2.1.3 — Komposisi Subgraph dan Penjelasan Cypher

Kueri yang digunakan beserta penjelasannya:

```
// (a) Komposisi simpul
MATCH (n)
RETURN labels(n)[0] AS nodeType, count(n) AS nodeCount
ORDER BY nodeCount DESC;

// (b) Komposisi relasi
MATCH ()-[r]->()
RETURN type(r) AS relType, count(r) AS relCount
ORDER BY relCount DESC;
```

Penjelasan (a). Pola `MATCH (n)` tanpa label mencocokkan seluruh simpul graf. Fungsi `labels(n)` mengembalikan *daftar* label karena satu simpul di Neo4j boleh memiliki lebih dari satu label; indeks `[0]` mengambil label pertama, yang dalam graf ini selalu `Visitor` atau `Item`. Agregasi `count(n)` bekerja secara implisit berdasarkan kunci pengelompokan, yaitu seluruh kolom non-agregat pada `RETURN` — dalam hal ini `nodeType`. Neo4j tidak memerlukan `GROUP BY` eksplisit seperti SQL.

Penjelasan (b). Pola `()-[r]->()` memakai simpul anonim di kedua ujung sehingga seluruh relasi berarah tercakup tanpa membatasi label. `type(r)` mengembalikan nama tipe relasi — berbeda dari `labels()`, fungsi ini mengembalikan satu string karena sebuah relasi di Neo4j hanya boleh memiliki tepat satu tipe. Setelah write-back similarity dijalankan, kueri yang sama akan memunculkan tipe keempat, yaitu `SIMILAR`.

Hasil yang diharapkan (setelah perbaikan Temuan 1 dijalankan):

Simpul	Jumlah	Relasi	Jumlah
Item	9.137	VIEWED	23.349
Visitor	374	ADDED_TO_CART	1.482
Total	**9.511**	BOUGHT	1.055
		Total	**25.886**

Struktur ini adalah graf bipartite: tidak ada relasi Visitor–Visitor maupun Item–Item sebelum tahap similarity. Rasio relasi terhadap simpul (25.886 / 9.511 ≈ 2,7) menunjukkan graf yang sangat renggang, sejalan dengan sparsity matriks 99,57%.

F.3 Soal 2.1.4 dan 2.1.5 — Karakteristik VSM

VSM simpul User. Setiap pengunjung direpresentasikan sebagai vektor baris pada ruang berdimensi item. Matriks berukuran 374 × 9.137 (3.417.238 sel) dengan 14.568 elemen non-nol, sparsity 99,57%, dan rata-rata 39,0 item unik per pengunjung. Nilai sel adalah bobot interaksi terakumulasi (1 untuk *view*, 2 untuk *addtocart*, 3 untuk *transaction*). Pada proyeksinya, dua pengunjung dihubungkan bila berbagi item yang sama; bobot edge adalah cosine similarity antarvektor.

VSM simpul Item. Matriks adalah transpose dari matriks User, berukuran 9.137 × 374 dengan jumlah elemen non-nol yang persis sama (14.568) dan sparsity yang persis sama (99,57%). Rata-rata hanya 1,59 pengunjung per item — artinya mayoritas produk hanya disentuh oleh satu pengunjung. Pada proyeksinya, dua item dihubungkan bila pernah diakses pengunjung yang sama (*co-occurrence*), dan banyak simpul menjadi terisolasi akibat rata-rata derajat yang sangat rendah tersebut.

> **Poin yang membedakan jawaban baik:** sparsity kedua matriks identik karena yang satu transpose dari yang lain. Yang berbeda bukan kerapatan matriksnya, melainkan **kerapatan graf proyeksi hasilnya** — proyeksi User menghasilkan komunitas yang terhubung rapat, proyeksi Item menghasilkan jaringan terfragmentasi dengan banyak komponen terpisah.

F.4 Soal 2.1.6 — Proyeksi Mana yang Lebih Baik

Jawaban: **proyeksi simpul User**, unggul pada efisiensi maupun efektivitas.

Parameter	Proyeksi User	Proyeksi Item
Jumlah simpul	374	9.137
Dimensi fitur	9.137	374
Matriks similarity	374 × 374 = 139.876 sel	9.137 × 9.137 = 83.484.869 sel
Kompleksitas	$O(\|U\|^2 \cdot \|I\|) \approx 1{,}28 \times 10^9$	$O(\|I\|^2 \cdot \|U\|) \approx 3{,}12 \times 10^{10}$
Memori (dense float64)	± 1,1 MB	± 637 MB
Derajat rata-rata	Tinggi (39 item per user)	Rendah (1,59 user per item)
Kesesuaian tugas	Langsung mendukung user-based KNN	Perlu langkah agregasi tambahan

Efisiensi. Rasio 83.484.869 / 139.876 ≈ 597 pada ukuran matriks, dan ≈ 24,4× pada estimasi FLOPs. Selisih ini muncul karena biaya komputasi similarity tumbuh kuadratik terhadap jumlah objek yang diproyeksikan, sedangkan dimensi fitur hanya berkontribusi linear. Karena |I| / |U| ≈ 24,4, proyeksi pada sisi yang jumlah objeknya lebih sedikit selalu lebih murah.

Efektivitas. Soal 2.2 meminta rekomendasi untuk *satu pengguna*. Proyeksi User menempatkan tetangga pengguna tersebut hanya satu langkah traversal dari simpul target, sehingga rekomendasi dapat langsung dibentuk dari preferensi tetangga. Pada proyeksi Item, prosedur yang sama memerlukan dua tahap: mencari item milik target, lalu mengagregasi item-item yang mirip dengannya. Ditambah lagi, derajat rata-rata 1,59 membuat sebagian besar simpul item terisolasi sehingga sinyal co-occurrence lemah.

Batas keberlakuan kesimpulan ini — penting bila dosen menguji lebih jauh: keunggulan ini berlaku karena pada subgraph ini |U| < |I|. Pada katalog yang kecil dengan jumlah pengguna besar (misalnya 10 juta pengguna, 500 produk), kesimpulannya berbalik dan proyeksi Item menjadi yang lebih efisien — inilah alasan Amazon secara historis memilih *item-based collaborative filtering*. Proyeksi Item juga lebih stabil terhadap waktu karena katalog produk berubah lebih lambat daripada populasi pengguna.

F.5 Soal 2.1.7 — Similarity dan Penulisan Kembali ke Graf

Record acuan: Visitor `1293068`. Metode: cosine similarity antara vektor target dengan seluruh baris matriks VSM User.

(a) Cypher untuk mengambil 5 record similarity tertinggi:

```
MATCH (u1:Visitor {visitorId: 1293068})-[r:SIMILAR]->(u2:Visitor)
RETURN u1.visitorId AS sourceUser,
       u2.visitorId AS similarUser,
       r.score      AS similarityScore
ORDER BY r.score DESC
LIMIT 5;
```

(b) Penulisan kembali nilai similarity ke graf:

```
UNWIND $batch AS rec
MATCH (u1:Visitor {visitorId: rec.source})
MATCH (u2:Visitor {visitorId: rec.target})
MERGE (u1)-[r:SIMILAR]->(u2)
SET   r.score = rec.score
RETURN count(r) AS updatedCount;
```

Bentuk di atas sudah memisahkan `MERGE` dari penetapan properti sesuai catatan pada Bagian C.3, sehingga idempoten bila dijalankan ulang. Hasil kueri verifikasi tercantum pada Temuan 2.

Mengapa similarity dihitung di Python, bukan di Cypher? Jawaban yang tepat bila ditanya: Cypher tidak memiliki operator aljabar vektor bawaan untuk perhitungan massal, sedangkan `sklearn.metrics.pairwise.cosine_similarity` memanfaatkan BLAS teroptimasi di atas matriks sparse. Alternatif yang murni di dalam database adalah pustaka Graph Data Science (GDS) melalui `gds.nodeSimilarity.write` atau `gds.knn.write`, yang menjalankan perhitungan di dalam Neo4j dan menulis hasilnya langsung sebagai relasi.

F.6 Soal 2.1.8 — Kondisi Graf Sebelum dan Sesudah

Aspek	Sebelum	Sesudah
Jenis graf	Bipartite (two-mode network)	Bipartite diperkaya dengan lapisan monopartite
Label simpul	Visitor, Item	Visitor, Item (tidak berubah)
Tipe relasi	VIEWED, ADDED_TO_CART, BOUGHT	Ditambah SIMILAR berbobot `score`
Relasi Visitor–Visitor	Tidak ada	Ada, 5 relasi dari simpul acuan
Jarak antar-user mirip	2 hop (melalui Item perantara)	1 hop (langsung)
Total relasi	25.886	25.891

Cypher kondisi sebelum:

```
MATCH (v:Visitor {visitorId: 1293068})-[r]->(i:Item)
RETURN type(r) AS relType, count(r) AS interactionCount
ORDER BY interactionCount DESC;

// Pembuktian ketiadaan relasi antar-Visitor:
MATCH (a:Visitor)-[r]->(b:Visitor) RETURN count(r) AS relasiAntarVisitor;  // -> 0
```

Cypher kondisi sesudah:

```
MATCH (v1:Visitor {visitorId: 1293068})-[r:SIMILAR]->(v2:Visitor)
RETURN v1.visitorId AS focalUser, v2.visitorId AS neighborUser, r.score AS similarityScore
ORDER BY r.score DESC;
```

Makna transformasinya. Sebelum similarity dihitung, menemukan pengguna serupa menuntut traversal dua langkah dan agregasi runtime: `(v1)-[:VIEWED]->(i)<-[:VIEWED]-(v2)`. Setelah nilai similarity dimaterialisasi sebagai edge, operasi yang sama menjadi pembacaan tetangga langsung berbiaya konstan. Ini adalah pertukaran klasik antara ruang penyimpanan dan waktu kueri: graf menyimpan hasil komputasi agar tidak dihitung ulang setiap permintaan rekomendasi.

F.7 Soal 2.2.1 — Rekomendasi KNN

(a) Skrip Python. Algoritmanya adalah *user-based collaborative filtering* dengan agregasi berbobot:

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

# 1. Pemilihan target user yang dapat direproduksi
np.random.seed(17)
kandidat = sorted(set(sub_df["visitorid"]) & set(df_p2["visitorid"]))
target_user = int(np.random.choice(kandidat))

# 2. Cari k tetangga terdekat berdasarkan cosine similarity
target_idx = user_map[target_user]
sims = cosine_similarity(mat_user[target_idx], mat_user).flatten()
neighbors = [i for i in np.argsort(sims)[::-1] if i != target_idx][:5]

# 3. Agregasi skor kandidat dari item milik tetangga
user_items = set(sub_df[sub_df["visitorid"] == target_user]["itemid"])
bobot = {"view": 1.0, "addtocart": 2.0, "transaction": 3.0}
skor = {}

for idx in neighbors:
    sim = float(sims[idx])
    for _, row in sub_df[sub_df["visitorid"] == users[idx]].iterrows():
        item = int(row["itemid"])
        if item not in user_items:                      # buang item yang sudah dikenal
            skor[item] = skor.get(item, 0.0) + sim * bobot.get(row["event"], 1.0)

# 4. Peringkat akhir
rekomendasi = sorted(skor.items(), key=lambda x: x[1], reverse=True)[:3]
```

Formula penyekorannya: skor prediksi untuk item *i* adalah jumlah dari (similarity tetangga *v* terhadap target) dikalikan (bobot interaksi tetangga *v* terhadap item *i*), untuk seluruh tetangga dalam himpunan k. Filter `item not in user_items` penting: tanpa itu sistem akan merekomendasikan barang yang sudah dilihat pengguna, yang tidak bernilai sebagai iklan.

(b) Tiga item yang diiklankan untuk k = 5:

Peringkat	Item ID	Skor prediksi	Status terhadap P2
1	38965	3,5862	Tidak diakses
2	241555	3,5709	**Hit** — benar diakses pada periode testing
3	265571	2,6602	Tidak diakses

Selisih skor peringkat 1 dan 2 hanya 0,0153 (0,4%), sehingga urutan keduanya praktis setara. Peringkat 3 turun tajam ke 2,6602,

menandakan hanya dua item yang benar-benar didukung konsensus beberapa tetangga.

F.8 Soal 2.2.2 — Hit@5 dan NDCG@5

Ground truth. Visitor 1293068 pada Fold Testing P2 mengakses 7 item unik: 9877, 37115, 96377, 98747, 239006, 241555, 283115.

Vektor relevansi Top-5: [0, 1, 0, 1, 0] — hit pada peringkat 2 (item 241555) dan peringkat 4 (item 239006).

(a) Hit@5 = 1. Hit@5 didefinisikan biner: bernilai 1 bila sedikitnya satu item relevan muncul dalam lima rekomendasi teratas, dan 0 bila tidak ada sama sekali. Karena terdapat dua item relevan, nilainya 1.

> **Antisipasi pertanyaan:** bila dosen mempertanyakan mengapa bukan 0,4 — jelaskan bahwa 2/5 = 0,4 adalah **Precision@5**, metrik yang berbeda. Hit@5 (sering disebut *Hit Rate*) bersifat biner per pengguna dan baru menjadi pecahan ketika dirata-ratakan terhadap banyak pengguna. Karena evaluasi di sini hanya untuk satu pengguna, nilainya tentu 0 atau 1. Menyebutkan keduanya sekaligus adalah jawaban paling aman.

(b) NDCG@5 = 0,3601. Perhitungan langkah demi langkah:

```
DCG@5  = 0/log2(2) + 1/log2(3) + 0/log2(4) + 1/log2(5) + 0/log2(6)
       = 0 + 0,6309 + 0 + 0,4307 + 0
       = 1,0616

IDCG@5 = 1/log2(2) + 1/log2(3) + 1/log2(4) + 1/log2(5) + 1/log2(6)
       = 1,0000 + 0,6309 + 0,5000 + 0,4307 + 0,3869
       = 2,9485

NDCG@5 = 1,0616 / 2,9485 = 0,3601
```

IDCG dihitung dari skenario ideal: seluruh lima posisi teratas terisi item relevan. Skenario tersebut memang mungkin terjadi di sini karena ground truth memuat 7 item, lebih banyak daripada k = 5, sehingga pembatas `min(|ground truth|, 5)` bernilai 5.

Interpretasi. Nilai 0,3601 berarti sistem mencapai sekitar 36% dari kualitas pemeringkatan ideal. Untuk *implicit feedback* e-commerce dengan sparsity 99,57% dan tanpa pelatihan model apa pun, angka ini tergolong wajar hingga baik. Penempatan satu hit pada peringkat 2 berkontribusi 0,6309 dari total 1,0616 (59%), memperlihatkan karakter NDCG yang memberi penghargaan besar pada posisi atas.

G. Antisipasi Pertanyaan Acak di Kelas

Naskah soal menyebut akan ada pertanyaan acak pada sesi kelas. Berikut pertanyaan yang paling mungkin muncul beserta inti jawabannya.

Pertanyaan	Inti jawaban
Mengapa `MERGE` untuk simpul tetapi `CREATE` untuk relasi?	Simpul harus unik; interaksi tidak. Satu pengguna dapat melihat produk yang sama berkali-kali, dan setiap peristiwa perlu menyimpan `timestamp`-nya sendiri.
Mengapa sparsity User dan Item sama persis?	Matriks Item adalah transpose matriks User. Jumlah elemen non-nol dan total sel tidak berubah oleh transpose.
Mengapa 25.886 baris CSV tetapi nnz hanya 14.568?	`csr_matrix` menjumlahkan bobot pada koordinat yang sama; 14.568 adalah pasangan unik user–item, 25.886 adalah jumlah peristiwa.
Dari mana bobot 1, 2, 3 berasal?	Asumsi ordinal pada *implicit feedback*: transaksi menyiratkan minat lebih kuat daripada penambahan ke keranjang, yang lebih kuat daripada sekadar melihat. Bukan hasil kalibrasi, melainkan pilihan desain yang dapat diganti.
Mengapa cosine, bukan Jaccard atau Pearson?	Cosine menangani vektor berbobot dan bersifat tahan terhadap perbedaan jumlah interaksi antarpengguna (hanya arah vektor yang dibandingkan, bukan panjangnya). Jaccard hanya cocok untuk data biner; Pearson memerlukan pemusatan rata-rata yang kurang bermakna pada implicit feedback.
Mengapa k = 5?	Ditentukan oleh naskah soal. Secara umum, k kecil menghasilkan rekomendasi yang spesifik tetapi mudah terpengaruh derau; k besar menghasilkan rekomendasi yang lebih umum dan cenderung ke item populer.
Apa yang terjadi bila database tidak direset sebelum impor ulang?	Simpul tidak terduplikasi karena `MERGE` dan constraint unik, tetapi seluruh relasi akan terduplikasi karena menggunakan `CREATE`, sehingga jumlah interaksi menjadi berlipat.
Mengapa similarity tidak dihitung dengan Cypher saja?	Cypher tidak memiliki operator aljabar vektor massal; alternatif di dalam database adalah pustaka GDS (`gds.nodeSimilarity`, `gds.knn`).
Bagaimana menangani pengguna baru tanpa riwayat (*cold start*)?	Vektornya nol sehingga cosine similarity tidak terdefinisi atau bernilai nol terhadap semua orang. Penanganannya memakai rekomendasi berbasis popularitas atau konten hingga riwayat interaksi terkumpul.
Mengapa evaluasi memakai pembagian berbasis waktu, bukan acak?	Pembagian acak membocorkan informasi masa depan ke data latih. Pemisahan berbasis periode meniru kondisi produksi: model hanya boleh melihat masa lalu untuk memprediksi masa depan.

H. Daftar Periksa Sebelum Pengumpulan

No	Tindakan	Prioritas
1	Sisipkan sel impor Neo4j sebelum sel komposisi, lalu jalankan ulang notebook dari atas sampai bawah	**Tinggi**
2	Pastikan output sel komposisi subgraph (2.1.3) menampilkan angka, bukan kosong	**Tinggi**
3	Pastikan output verifikasi relasi SIMILAR (2.1.7b) menampilkan 5 baris	**Tinggi**
4	Tambahkan sel kode pemilihan user acak dengan `np.random.seed(17)` dan selaraskan `TARGET_USER` pada skrip	**Tinggi**
5	Tambahkan sel yang memperlihatkan cara `events_subgraph.csv` dibentuk dari P1	**Sedang**
6	Ubah `MERGE ... {score}` menjadi `MERGE ... SET r.score` agar idempoten	**Sedang**
7	Selaraskan angka pada README dengan output notebook setelah eksekusi ulang	**Sedang**
8	Pertimbangkan memperluas plot proyeksi Item agar tidak hanya 8 item	**Rendah**
9	Pastikan `data/events.csv` (94 MB) tidak ikut terdorong ke GitHub bila melewati batas ukuran berkas	**Sedang**
10	Unggah notebook terisi output, skrip, dan dataset hasil filter ke GitHub; kumpulkan tautannya di LMS SiDia	**Tinggi**

Dokumen panduan disusun berdasarkan pemeriksaan langsung terhadap berkas notebook, skrip Python, dan berkas keluaran pada direktori `UTS GRAF/` per 23 Oktober 2026.